

Permutation-aligned averaging: hidden-unit symmetry and the linear assignment reduction

Companion proof for: [pleyva2004/communication-efficient-learning-of-deep-networks](#)

Improvement slug: `permutation-aligned-averaging`

Abstract

The math deep dive (§7b) attributes Figure 1’s loss barrier between independently-initialized networks to the permutation symmetry of a hidden layer, and improvement T.1 of `05-improvements.tex` proposes to remove that barrier before averaging by re-aligning each client model into a common basin. This note makes both halves rigorous. We prove (i) that for a one-hidden-layer network $h(x) = W_2 \sigma(W_1 x + b_1)$ with σ applied elementwise, the reparametrization $W_1 \mapsto PW_1$, $b_1 \mapsto Pb_1$, $W_2 \mapsto W_2 P^\top$ by *any* permutation matrix P leaves the function h *exactly* unchanged, hence leaves any loss unchanged; and (ii) that minimizing the Frobenius distance $\|W_1 - PW_1'\|_F^2$ over permutation matrices is equivalent to the linear assignment problem of maximizing $\sum_i \langle (W_1)_{i,:}, (W_1')_{P(i),:} \rangle$, because the two squared-norm terms are permutation-invariant and drop out — a problem solvable exactly in $O(H^3)$ time by the Hungarian algorithm. Part (i) follows from $P^\top P = I$ together with the fact that an elementwise nonlinearity commutes with a coordinate permutation; part (ii) is a direct expansion of the Frobenius inner product. Together they justify “align then average” (weight-matching, in the sense of Git Re-Basin, Ainsworth et al. 2023) as an exact symmetry reduction with a tractable matching step.

Setup and notation

We work with a single-hidden-layer feed-forward network with H hidden units. Fix an input dimension d_{in} , a hidden width H , and an output dimension d_{out} . The network is parametrized by a first-layer weight matrix $W_1 \in \mathbb{R}^{H \times d_{\text{in}}}$, a hidden bias $b_1 \in \mathbb{R}^H$, and a second-layer weight matrix $W_2 \in \mathbb{R}^{d_{\text{out}} \times H}$. A fixed scalar activation $\phi : \mathbb{R} \rightarrow \mathbb{R}$ (e.g. ReLU $\phi(t) = \max(t, 0)$, or tanh) is applied *elementwise*: for a vector $z = (z_1, \dots, z_H)^\top \in \mathbb{R}^H$ we write $\sigma(z) = (\phi(z_1), \dots, \phi(z_H))^\top$, so $\sigma : \mathbb{R}^H \rightarrow \mathbb{R}^H$ acts coordinatewise with the *same* scalar map ϕ in every coordinate. The forward map is

$$h(x) = W_2 \sigma(W_1 x + b_1), \quad x \in \mathbb{R}^{d_{\text{in}}}.$$

The deep dive’s notation in §7b uses $W_2 \sigma(W_1 x + b_1)$ for the 2NN with 200 hidden units per layer; here H plays the role of that hidden width.

A *permutation matrix* $P \in \mathbb{R}^{H \times H}$ is the matrix of a bijection $\pi : \{1, \dots, H\} \rightarrow \{1, \dots, H\}$, defined by $P_{ij} = 1$ if $j = \pi(i)$ and $P_{ij} = 0$ otherwise; equivalently $P_{ij} = \mathbf{1}\{j = \pi(i)\}$. We write $P(\cdot)$ for the induced reparametrization of the parameter triple and Π_H for the set of all $H \times H$ permutation matrices, a finite group of order $H!$ under matrix multiplication. We use the Frobenius inner product $\langle A, B \rangle_F = \sum_{ij} A_{ij} B_{ij} = \text{tr}(A^\top B)$ and the induced norm $\|A\|_F^2 = \langle A, A \rangle_F$. Rows are written $(W)_{i,:}$ and columns $(W)_{:,j}$. The linear assignment problem (LAP) on a cost/benefit matrix $C \in \mathbb{R}^{H \times H}$ asks for the bijection π optimizing $\sum_i C_{i,\pi(i)}$.

Definition 1 (Working notation). For a permutation π with matrix P ($P_{ij} = \mathbf{1}\{j = \pi(i)\}$):

- **Action on a vector.** For $z \in \mathbb{R}^H$, the i -th coordinate of Pz is $(Pz)_i = \sum_j P_{ij}z_j = z_{\pi(i)}$. Thus P *gathers*: it places old coordinate $\pi(i)$ into new slot i .
- **Orthogonality.** Each row and each column of P has exactly one 1, so the columns of P are the standard basis vectors in some order; hence $P^\top P = PP^\top = I_H$, i.e. $P^{-1} = P^\top$ and P is orthogonal.
- **Elementwise nonlinearity.** $\sigma : \mathbb{R}^H \rightarrow \mathbb{R}^H$, $\sigma(z)_i = \phi(z_i)$, with the *same* scalar ϕ in every coordinate.
- **Reparametrization.** P acts on the parameter triple by $W_1 \mapsto PW_1$, $b_1 \mapsto Pb_1$, $W_2 \mapsto W_2P^\top$.
- **Loss.** For any data distribution $\mathcal{D}$ over (x, y) and any per-example loss ℓ , the population (or empirical) risk of a parameter set $\theta = (W_1, b_1, W_2)$ is $L(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} \ell(h_\theta(x), y)$, where h_θ is the forward map above.

Theorem

Theorem 2 (Hidden-unit permutation symmetry and the assignment reduction). *Let $h(x) = W_2 \sigma(W_1 x + b_1)$ be the one-hidden-layer network of the Setup, with σ applied elementwise via a fixed scalar ϕ . Then:*

(i) **Exact functional symmetry.** *For every permutation matrix $P \in \Pi_H$, the reparametrized network*

$$\widetilde{W}_1 = PW_1, \quad \widetilde{b}_1 = Pb_1, \quad \widetilde{W}_2 = W_2P^\top$$

computes the identical function: $\widetilde{W}_2 \sigma(\widetilde{W}_1 x + \widetilde{b}_1) = h(x)$ for all $x \in \mathbb{R}^{d_{\text{in}}}$. Consequently, for any data distribution and any loss ℓ , the risk is unchanged: $L(PW_1, Pb_1, W_2P^\top) = L(W_1, b_1, W_2)$.

(ii) **Frobenius alignment $\equiv$ linear assignment.** *Let $W_1, W'_1 \in \mathbb{R}^{H \times d_{\text{in}}}$ be two first-layer weight matrices. Then for every $P \in \Pi_H$ with associated permutation π ,*

$$\|W_1 - PW'_1\|_F^2 = \|W_1\|_F^2 + \|W'_1\|_F^2 - 2 \sum_{i=1}^H \langle (W_1)_{i,:}, (W'_1)_{\pi(i),:} \rangle,$$

where the first two terms are constants independent of P . Hence

$$\arg \min_{P \in \Pi_H} \|W_1 - PW'_1\|_F^2 = \arg \max_{P \in \Pi_H} \sum_{i=1}^H \langle (W_1)_{i,:}, (W'_1)_{\pi(i),:} \rangle.$$

The right-hand side is a linear assignment problem on the $H \times H$ benefit matrix $C_{i,j} = \langle (W_1)_{i,:}, (W'_1)_{j,:} \rangle$, and it can be solved exactly in $O(H^3)$ time (the Hungarian algorithm of Kuhn 1955 and Munkres 1957).

Proof

Proof. We prove the two parts in turn. Throughout, $P \in \Pi_H$ is the matrix of a bijection π with $P_{ij} = \mathbf{1}\{j = \pi(i)\}$.

Part (i): exact functional symmetry.

Step 1 — P is orthogonal, $P^\top P = I_H$. Fix indices $i, k \in \{1, \dots, H\}$ and compute the (i, k) entry of $P^\top P$. By definition of matrix multiplication and of P ,

$$(P^\top P)_{ik} = \sum_{m=1}^H (P^\top)_{im} P_{mk} = \sum_{m=1}^H P_{mi} P_{mk} = \sum_{m=1}^H \mathbf{1}\{i = \pi(m)\} \mathbf{1}\{k = \pi(m)\}.$$

Each summand is 1 exactly when $\pi(m) = i$ and $\pi(m) = k$ simultaneously, which requires $i = k$. Because π is a bijection, for each i there is exactly one index $m = \pi^{-1}(i)$ with $\pi(m) = i$. Hence the sum equals 1 if $i = k$ and 0 otherwise, i.e. $(P^\top P)_{ik} = \delta_{ik}$, so $P^\top P = I_H$. (The same computation with the roles of rows and columns swapped gives $PP^\top = I_H$; we only need $P^\top P = I_H$ below.) In particular $P^\top = P^{-1}$.

Step 2 — an elementwise nonlinearity commutes with a coordinate permutation. We claim that for every $z \in \mathbb{R}^H$,

$$\sigma(Pz) = P\sigma(z).$$

Both sides are vectors in $\mathbb{R}^H$; we compare their i -th coordinates. Recall from the working notation that $(Pz)_i = \sum_j P_{ij} z_j = z_{\pi(i)}$, since the only nonzero entry in row i of P is $P_{i,\pi(i)} = 1$. Applying σ coordinatewise,

$$(\sigma(Pz))_i = \phi((Pz)_i) = \phi(z_{\pi(i)}).$$

On the other side, $\sigma(z)$ is the vector with j -th coordinate $\phi(z_j)$, and applying P to it gathers coordinate $\pi(i)$ into slot i :

$$(P\sigma(z))_i = \sum_j P_{ij} \sigma(z)_j = \sigma(z)_{\pi(i)} = \phi(z_{\pi(i)}).$$

The two expressions agree for every i , so $\sigma(Pz) = P\sigma(z)$. The only property of σ used is that it acts coordinatewise with one and the same scalar ϕ in every coordinate; this is exactly what “ σ applied elementwise” means, and it is what makes the permutation slide through the nonlinearity. (If different coordinates used different scalar functions, this step would fail.)

Step 3 — assemble the forward pass. Let $\widetilde{W}_1 = PW_1$, $\widetilde{b}_1 = Pb_1$, $\widetilde{W}_2 = W_2 P^\top$, and fix an arbitrary input $x \in \mathbb{R}^{d_{\text{in}}}$. The reparametrized pre-activation is

$$\widetilde{W}_1 x + \widetilde{b}_1 = (PW_1)x + (Pb_1) = P(W_1 x) + Pb_1 = P(W_1 x + b_1),$$

using only distributivity of matrix multiplication and associativity $(PW_1)x = P(W_1 x)$. Write $z := W_1 x + b_1 \in \mathbb{R}^H$ for the original pre-activation, so the reparametrized pre-activation is Pz . Pushing through σ and then the second layer:

$$\widetilde{W}_2 \sigma(\widetilde{W}_1 x + \widetilde{b}_1) = (W_2 P^\top) \sigma(Pz) \stackrel{\text{Step 2}}{=} (W_2 P^\top) (P\sigma(z)) = W_2 (P^\top P) \sigma(z) \stackrel{\text{Step 1}}{=} W_2 \sigma(z),$$

where the middle equality used associativity to regroup $(W_2 P^\top)(P\sigma(z)) = W_2 (P^\top P) \sigma(z)$, and the last used $P^\top P = I_H$ from Step 1. But $W_2 \sigma(z) = W_2 \sigma(W_1 x + b_1) = h(x)$ by definition. Since x was arbitrary,

$$\widetilde{W}_2 \sigma(\widetilde{W}_1 x + \widetilde{b}_1) = h(x) \quad \text{for all } x,$$

i.e. the two parameter triples realize the *same* function $\mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$.

Step 4 — loss invariance. Loss depends on the parameters only through the function they compute, because each example’s loss $\ell(h_\theta(x), y)$ is a function of the network *output* $h_\theta(x)$. Let $\theta = (W_1, b_1, W_2)$ and $\tilde{\theta} = (PW_1, Pb_1, W_2 P^\top)$. By Step 3, $h_{\tilde{\theta}}(x) = h_\theta(x)$ for every x , so $\ell(h_{\tilde{\theta}}(x), y) = \ell(h_\theta(x), y)$ for every (x, y) . Taking expectation over $(x, y) \sim \mathcal{D}$ (or the finite empirical average

over a fixed sample, which is the special case where $\mathcal{D}$ is the empirical distribution) and using monotonicity/linearity of expectation on a pointwise identity,

$$L(\tilde{\theta}) = \mathbb{E}_{(x,y)} \ell(h_{\tilde{\theta}}(x), y) = \mathbb{E}_{(x,y)} \ell(h_{\theta}(x), y) = L(\theta).$$

Hence $L(PW_1, Pb_1, W_2P^\top) = L(W_1, b_1, W_2)$, proving (i). As a structural corollary, the map $\theta \mapsto P(\theta)$ permutes the parameter space within a single level set of L ; ranging over all $P \in \Pi_H$ produces an orbit of (generically) $H!$ distinct parameter points that all attain the identical loss, matching the deep dive's $(200!)^2$ orbit count for the two-hidden-layer 2NN (one factor of $H!$ per hidden layer).

Part (ii): Frobenius alignment reduces to linear assignment.

Step 1 — expand the Frobenius norm. For any matrices A, B of the same shape, $\|A - B\|_F^2 = \langle A - B, A - B \rangle_F$. Expanding the inner product by bilinearity,

$$\|A - B\|_F^2 = \langle A, A \rangle_F - \langle A, B \rangle_F - \langle B, A \rangle_F + \langle B, B \rangle_F = \|A\|_F^2 - 2\langle A, B \rangle_F + \|B\|_F^2,$$

where we used symmetry $\langle A, B \rangle_F = \langle B, A \rangle_F$ of the (real) Frobenius inner product. Apply this with $A = W_1$ and $B = PW'_1$:

$$\|W_1 - PW'_1\|_F^2 = \|W_1\|_F^2 - 2\langle W_1, PW'_1 \rangle_F + \|PW'_1\|_F^2. \quad (*)$$

Step 2 — the third term is permutation-invariant. Using $\|M\|_F^2 = \text{tr}(M^\top M)$ and the cyclic property of the trace, with $P^\top P = I_H$ from Part (i) Step 1,

$$\|PW'_1\|_F^2 = \text{tr}((PW'_1)^\top (PW'_1)) = \text{tr}(W_1'^\top P^\top P W'_1) = \text{tr}(W_1'^\top W'_1) = \|W'_1\|_F^2.$$

(Geometrically: P is orthogonal, so left-multiplication by P is an isometry and preserves the Frobenius norm; concretely, the rows of PW'_1 are exactly the rows of W'_1 in the order prescribed by π , and reordering rows does not change the sum of squared entries.) The first term $\|W_1\|_F^2$ does not involve P at all. Therefore in $(*)$ both squared-norm terms are constants independent of P , and

$$\|W_1 - PW'_1\|_F^2 = \underbrace{\|W_1\|_F^2 + \|W'_1\|_F^2}_{\text{constant in } P} - 2\langle W_1, PW'_1 \rangle_F. \quad (**)$$

Step 3 — rewrite the cross term as a sum of row inner products. Expand the Frobenius inner product entrywise and substitute $(PW'_1)_{ik} = \sum_m P_{im}(W'_1)_{mk} = (W'_1)_{\pi(i),k}$ (only $m = \pi(i)$ contributes):

$$\langle W_1, PW'_1 \rangle_F = \sum_{i=1}^H \sum_{k=1}^{d_{\text{in}}} (W_1)_{ik} (PW'_1)_{ik} = \sum_{i=1}^H \sum_{k=1}^{d_{\text{in}}} (W_1)_{ik} (W'_1)_{\pi(i),k} = \sum_{i=1}^H \langle (W_1)_{i,:}, (W'_1)_{\pi(i),:} \rangle,$$

where the inner sum over k is precisely the Euclidean inner product of row i of W_1 with row $\pi(i)$ of W'_1 . Substituting into $(**)$ gives the claimed identity

$$\|W_1 - PW'_1\|_F^2 = \|W_1\|_F^2 + \|W'_1\|_F^2 - 2 \sum_{i=1}^H \langle (W_1)_{i,:}, (W'_1)_{\pi(i),:} \rangle.$$

Step 4 — minimization becomes maximization of the assignment objective. For each fixed pair (W_1, W'_1) the two squared-norm terms are a single additive constant $\kappa := \|W_1\|_F^2 + \|W'_1\|_F^2$ that does not depend on the choice of $P \in \Pi_H$. A function of the form $P \mapsto \kappa - 2S(P)$, with κ constant and

the coefficient $-2 < 0$, is minimized exactly where $S(P) := \sum_i \langle (W_1)_{i,:}, (W'_1)_{\pi(i),:} \rangle$ is maximized: for any $P, Q \in \Pi_H$,

$$\|W_1 - PW'_1\|_F^2 \leq \|W_1 - QW'_1\|_F^2 \iff \kappa - 2S(P) \leq \kappa - 2S(Q) \iff S(P) \geq S(Q).$$

Hence the set of minimizers of the left objective over the finite set Π_H equals the set of maximizers of S :

$$\arg \min_{P \in \Pi_H} \|W_1 - PW'_1\|_F^2 = \arg \max_{P \in \Pi_H} \sum_{i=1}^H \langle (W_1)_{i,:}, (W'_1)_{\pi(i),:} \rangle.$$

(Both $\arg \min$ and $\arg \max$ are over the finite set Π_H , so they are attained and nonempty; the equivalence is of the full minimizer/maximizer sets, not merely of a single optimizer.)

Step 5 — this is a linear assignment problem, solvable in $O(H^3)$. Define the benefit matrix $C \in \mathbb{R}^{H \times H}$ by $C_{ij} = \langle (W_1)_{i,:}, (W'_1)_{j,:} \rangle$, i.e. $C = W_1 W_1'^\top$ (entry (i, j) is the inner product of row i of W_1 with row j of W'_1). A permutation π is a perfect matching of the bipartite graph with left vertices $\{1, \dots, H\}$ (rows of W_1), right vertices $\{1, \dots, H\}$ (rows of W'_1), and edge weight $C_{i,j}$. The objective $S(P) = \sum_i C_{i,\pi(i)}$ is exactly the total weight of the matching π , so maximizing S over Π_H is the *maximum-weight perfect bipartite matching* (equivalently the linear assignment) problem on C . Maximization is converted to the standard minimization form by minimizing $C'_{ij} := (\max_{a,b} C_{ab}) - C_{ij} \geq 0$, an affine, π -independent transformation of the per-edge weights that adds the constant $H \cdot \max_{a,b} C_{ab}$ to every assignment’s total and hence preserves the optimizing π . The Hungarian algorithm (Kuhn 1955; Munkres 1957) solves this to global optimality — it returns a provably cost-minimizing assignment, not a heuristic — in $O(H^3)$ arithmetic operations on the $H \times H$ matrix C' . Therefore the Frobenius-optimal alignment P^* exists, is exactly computable, and is found in cubic time in the hidden width. This proves (ii). $\square$

Remark 3 (Aligning the whole network, not just W_1). By Part (i), applying P^* to the *whole* second model — $W'_1 \mapsto P^*W'_1$, $b'_1 \mapsto P^*b'_1$, $W'_2 \mapsto W'_2(P^*)^\top$ — leaves the second model’s function (and loss) unchanged while making its hidden units maximally row-aligned with the first model’s. The server may then average the aligned parameters, e.g. $\frac{1}{2}\theta + \frac{1}{2}P^*(\theta')$, blending hidden units that play matching roles rather than functionally unrelated ones. The matching above uses only W_1 ; one may instead build C from $[W_1 | b_1]$ or incorporate W_2 columns, which changes the cost matrix but not the reduction — it remains a single $O(H^3)$ assignment.

Discussion

What this gives us. Part (i) is an *exact* statement, holding for every x and every permutation, with no approximation, no smoothness assumption on ϕ beyond being a fixed scalar map, and no convexity: hidden-unit relabeling is a genuine symmetry of the loss surface, so every minimizer sits on an orbit of (generically) $H!$ equal-loss copies. This is the precise mechanism the math deep dive invokes in §7b to explain Figure 1’s loss barrier between independently-initialized models: two solutions found from different seeds generically land in *different* elements of the same orbit, so coordinate-wise averaging blends unrelated hidden units and produces a degraded interpolant. Part (ii) turns the proposed fix into something tractable and exactly solvable: “find the best relabeling” is not a search over $H!$ permutations but a polynomial-time linear assignment, because the permutation-invariant squared-norm terms drop out of the Frobenius objective, leaving a pure inner-product matching. Concretely this licenses the T.1 “align then average” procedure (weight-matching, in the sense of Git Re-Basin, Ainsworth et al. 2023): solve one $O(H^3)$ Hungarian assignment on $C = W_1 W_1'^\top$, apply the resulting symmetry (which by Part (i) costs nothing in

function value), then average. It is what relaxes FedAvg’s reliance on a *shared* per-round initialization (McMahan et al. 2017): the shared init was the cheap way to keep all clients in one orbit element, and explicit alignment provides an alternative that does not require it, which in principle permits larger local epochs E and fewer synchronization rounds in the drift-prone non-IID regime that motivates client-drift corrections such as SCAFFOLD (Karimireddy et al. 2020) and FedProx (Li et al. 2020).

What this does not give us. The theorem is purely about the *symmetry* and the *matching*; it says nothing about whether averaging two aligned models is actually good. Part (i) guarantees only that alignment is loss-preserving for the model being aligned, not that the post-alignment *average* has low loss — that is an empirical, landscape-dependent claim (the prototype’s barrier $+0.024 \rightarrow -0.023$ in T.1 is evidence, not a consequence of this proof). The matching in (ii) is optimal for the chosen *surrogate* objective $\|W_1 - PW'_1\|_F^2$ (raw first-layer weight distance); minimizing weight distance is a proxy for, and not the same as, minimizing the post-average loss barrier, so the “optimal” P^* is optimal for the surrogate only. Nothing here addresses multi-client aggregation: for $K > 2$ clients, jointly aligning all of them to a common reference so that pairwise matchings are mutually consistent is a separate combinatorial problem that the single $O(H^3)$ solve does not settle. Finally, the result is exact for the one-hidden-layer architecture stated; it does not by itself extend the function-preservation guarantee to convolutions, normalization layers, attention, or residual connections, each of which has its own (sometimes more constrained) symmetry group.

Where it would break. Part (i)’s Step 2 is the load-bearing hinge and it requires the nonlinearity to be *truly elementwise with the same scalar in every coordinate*: a per-channel parametrized activation (e.g. PReLU with distinct learned slopes per unit), a softmax or any cross-coordinate coupling, a LayerNorm/BatchNorm whose statistics mix coordinates, or a bias folded in non-permutably would all break $\sigma(Pz) = P\sigma(z)$, and the symmetry would no longer be exact (it would have to be carried by the matching layer’s statistics too). The clean second-layer cancellation also depends on the convention $W_2 \mapsto W_2 P^\top$ with the *same* P that acts on the first layer; mismatched or untied permutations would not produce $P^\top P = I$. For *deeper* networks, weight-matching is per-layer and greedy: aligning layer ℓ fixes a permutation that must be propagated to layer $\ell + 1$, and choosing each layer’s permutation by its own local Frobenius objective is not jointly optimal across layers — this is why activation-matching (aligning units by their responses on real data rather than by raw weight inner products) is generally needed for multilayer nets, and even then the joint problem is only solved approximately by coordinate descent over per-layer assignments (Ainsworth et al. 2023). Finally, the procedure is at odds with secure aggregation (Bonawitz et al. 2017): computing $C = W_1 W_1^\top$ and the matching requires the server to see individual client weight matrices, whereas secure aggregation is designed precisely so the server learns only the *sum* of updates and never any single client’s parameters; making alignment compatible with that privacy model (e.g. matching to a public reference, or matching under encryption) is a genuinely open, non-trivial obstacle, not a detail this proof resolves.

Empirical companion. This proof is the validation file for the improvement *permutation-aligned-averaging* in `05-improvements.tex` (proposal T.1). If a measurement-mode prototype also exists at `improvements/permutat` it serves as a sanity check, not as evidence — the theorem above is the load-bearing claim.